

Console Royale

카드 게임 패키지 프로그램

OOP 프로젝트 중간보고서

팀 명	3팀 : T.C.P.
구성원	권진형, 이성민, 이채호, 정재우
GitHub	https://github.com/rnfma0815/OOP
YouTube	https://www.youtube.com/watch?v=awqHtkyluWk

2026년 6월 10일

1. 프로젝트 개요

1.1 작품 주제 선정 배경

객체지향을 공부하던 중, '카드'라는 개념이 객체로 표현하기에 매우 적합하다는 점에 착안하였습니다. 처음에는 비교적 간단하게 구현할 수 있는 게임 제작을 목표로 하였으며, 여러 게임에서 공통으로 활용할 수 있고 객체지향 설계를 적용하기 적합한 소재로 트럼프 카드를 선택하였습니다. 카드 한 장을 객체로 만들고 이를 응용하면 블랙잭·클라이밍·메모리 게임 등 다양한 장르를 구현할 수 있으며, 그 과정에서 상속·다형성·추상 클래스 등 객체지향의 핵심 개념을 자연스럽게 익힐 수 있다고 판단하였습니다. 이를 통해 공통 기능 재사용, 클래스 구조 설계, 게임별 로직 분리를 목표로 프로젝트를 진행하였습니다.

1.2 프로젝트 개요

항목	내용
프로젝트명	Console Royale — C++ 객체지향 기반 콘솔 카드 게임 패키지
구현 게임	블랙잭 (1대1 대전) / 클라이밍 게임 (3세트 매치) / 메모리 게임
핵심 설계	공통 Card·Deck 구조를 세 게임에서 재사용, 게임별 로직 클래스 분리
난이도 시스템	Easy / Normal / Hard 3단계, 게임별 차별화 AI 알고리즘 적용
확장 방향	멀티플레이 기능 확장이 가능하도록 구조 설계

1.3 벤치마킹 자료

참고 게임	참고한 내용	실제 반영 사항
카지노 블랙잭	카드 배분 방식, 딜러 행동 기준	25장씩 배분·히든카드·5라운드 구조, 확률 기반 딜러 AI
그림 맞추기 (기억력 게임)	카드 뒤집기, 짝 찾기, 격자 보드	MemoryGame 보드 구조, Hard 무한 리필 메커니즘
대부호 게임 (PRESIDENT)	KING·QUEEN·JACK·JOKER 계층, 세트 매치	ClimbingGame Role enum, 3세트 매치, 미니맥스 AI

1.4 개발 환경

구분	항목
OS / HW	Windows 10/11 x64, 개인 PC
언어 / 표준	C++17
IDE	Visual Studio 2022
외부 라이브러리	표준 라이브러리만 사용 (STL: vector, string, algorithm, random 등)
콘솔 API	Windows.h (SetConsoleCursorPosition, SetConsoleTextAttribute 등)
입력 처리	conio.h (_getch)

2. 개발 프로그램 설명

2.1 파일 구성

프로젝트는 게임 로직과 화면 출력을 분리하는 구조로 설계되었으며, 총 18개의 .h / .cpp 쌍(+ State.h, main.cpp)으로 구성됩니다.

파일 (헤더 + 소스)	클래스 / 역할	비고
Game.h / Game.cpp	Game (추상 기반 클래스)	순수 가상함수 play()
GameManager.h / .cpp	GameManager	로비, 게임 선택, 난이도 관리
Player.h / .cpp	Player (추상)	순수 가상 calculateScore()
Dealer.h / .cpp	Dealer : Player (추상)	순수 가상 playTurn()
BlackjackPlayer.h / .cpp	BlackjackPlayer : Player	히든카드 기능 확장
BlackjackDealer.h / .cpp	BlackjackDealer : Dealer	난이도별 턴 로직
BlackjackDealerLogic.h / .cpp	BlackjackDealerLogic	딜러 AI 확률 계산 (Easy/Normal/Hard)
Blackjack.h / .cpp	Blackjack : Game	블랙잭 게임 진행
BlackjackView.h / .cpp	BlackjackView	블랙잭 화면 출력
MemoryGame.h / .cpp	MemoryGame : Game	기억력 게임 로직
MemoryGameView.h / .cpp	MemoryGameView	기억력 게임 화면 출력
ClimbingGame.h / .cpp	ClimbingGame : Game	클라이밍 게임 + 미니맥스 AI
ClimbingGameView.h / .cpp	ClimbingGameView	클라이밍 게임 화면 출력
Card.h / .cpp	Card	카드 데이터 (rank, suit)
Deck.h / .cpp	Deck	덱 생성, 셔플, 드로우
ConsoleHelper.h / .cpp	ConsoleHelper (namespace)	콘솔 공통 유틸 (커서, 색상)
InputHelper.h / .cpp	InputHelper (namespace)	키 입력 추상화
PauseMenuView.h / .cpp	PauseMenuView	일시정지 메뉴
State.h	State namespace (enum)	Difficulty, PauseResult, GameResult
main.cpp	진입점	GameManager 생성 후 run()

※ 상속 구조: Game ← Blackjack / MemoryGame / ClimbingGame, Player ← Dealer ← BlackjackDealer, Player ← BlackjackPlayer

2.2 UI/UX 설계

전체 인터페이스는 Windows 콘솔 기반으로, 방향키 + 엔터 방식의 메뉴를 사용합니다. ConsoleHelper를 통해서 이동과 색상 출력을 공통 처리합니다.

화면	주요 구성 요소	조작 방법
로비 메뉴	카드 ASCII 아트, Start / Options / Exit	↑ ↓ 이동 + Enter
게임 선택	게임 목록, Start / Difficulty / Description / Back	← → 전환, ↑ ↓ 메뉴
블랙잭	플레이어·딜러 카드 영역, 점수, 히든카드 선택	↑ ↓ + Enter, ESC 일시정지
기억력 게임	격자 보드 (Easy 4×2 / Normal 5×2 / Hard 6×2+리필)	방향키 커서 이동, Enter 선택
클라이밍 게임	손패 (KING/QUEEN/SERVANT/JOKER), 세트·라운드 스코어	↑ ↓ + Enter, ESC 일시정지
일시정지 메뉴	Resume / Restart / Exit to Menu	↑ ↓ + Enter, ESC 호출

2.3 클래스 상세

▶ Game — 추상 기반 클래스

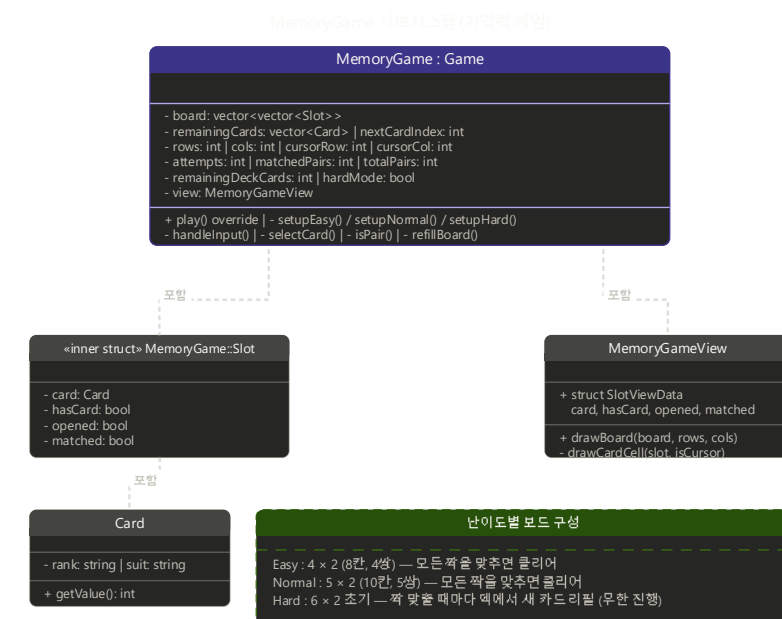
모든 게임의 공통 인터페이스를 정의합니다. play()를 순수 가상함수로 선언하여 세 게임이 반드시 구현하도록 강제하며, GameManager는 Game* 포인터 벡터로 세 게임을 다형적으로 관리합니다.

▶ Player / Dealer — 상속 계층

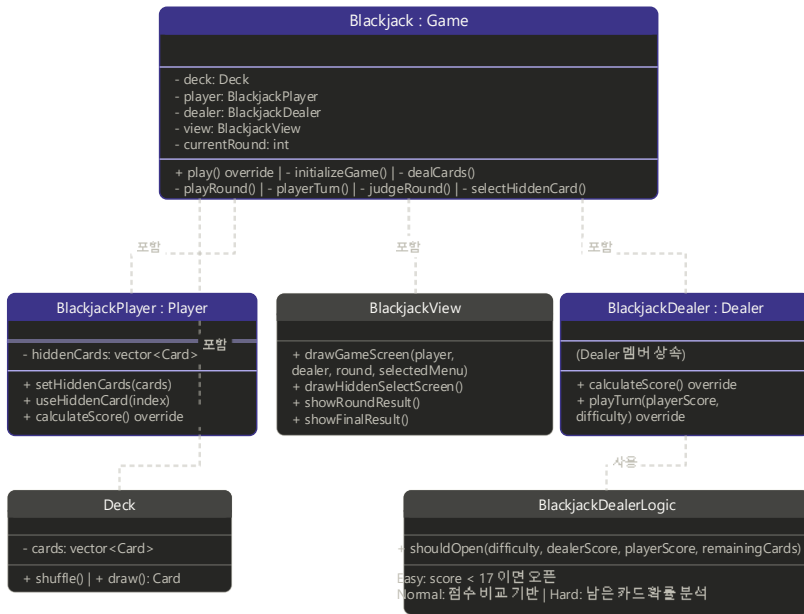
Player는 카드 더미(pileCards), 라운드 카드(roundCards), 공개 카드(openedCards)와 calculateScore()를 갖는 추상 클래스입니다. Dealer는 Player를 상속하고 playTurn()을 추가로 선언합니다. BlackjackPlayer와 BlackjackDealer가 각각을 상속하여 블랙잭 전용 로직을 구현합니다.

▶ Card / Deck — 공통 카드 구조

Card는 rank(A~2~10·J·Q·K)와 suit(Spade·Diamond·Heart·Clover) 두 멤버로 구성됩니다. Deck은 52장을 생성·서플하고 draw()로 한 장씩 반환합니다. 세 게임이 이 공통 구조를 재사용합니다.



Blackjack 서비스시스템 (블랙잭 게임)



ClimbingGame 서비스시스템 (클라이밍 게임)



2.4 게임 모드 및 멀티 요소

▶ 블랙잭 — 플레이어 vs 딜러 1 대 1 대전

블랙잭은 플레이어와 딜러가 52장의 카드를 가지고 1대1로 대전하는 구조입니다. 게임 시작 시 딜러에게 25장, 플레이어에게 25장, 나머지 2장은 플레이어의 히든카드로 배분됩니다. 5라운드에 걸쳐 라운드별 승점을 누적하며, 최종적으로 더 많은 라운드를 이긴 쪽이 승리합니다.

구분	내용
카드 배분	딜러 25장 / 플레이어 25장 / 히든카드 2장 (총 52장)
라운드 구조	총 5라운드 진행, 라운드별 승점 누적 후 최종 승자 결정
플레이어 행동	카드 오픈 (덱에서 1장 공개) / Stand (그대로 스톱) / 히든카드 사용
히든카드	보유한 2장 중 1장을 선택해 현재 손패에 추가하는 전략 요소
승패 판정	21 정확히 → 즉시 승리 / 버스트(21 초과) → 패배 / 21에 더 가까운 쪽 승리

▶ 클라이밍 게임 — 3 세트 매치

클라이밍 게임은 플레이어와 컴퓨터 AI가 카드를 동시에 선택하며 맞붙는 구조입니다. 하나의 매치는 3세트로 구성되며, 먼저 2세트를 획득한 쪽이 최종 승리합니다(Best of 3). 각 세트는 KING·QUEEN·SERVANT·JOKER 4장으로 4라운드를 진행하며, 카드는 한 번 사용하면 소진됩니다.

구분	내용
매치 구조	3세트 매치 (Best of 3) — 먼저 2세트 획득 시 최종 승리
세트 구조	4라운드 1세트 — KING·QUEEN·SERVANT·JOKER 각 1장씩 사용
카드 계층	KING (최강) > QUEEN > SERVANT / JOKER: KING에게 승, SERVANT에게 패
선택 방식	플레이어와 컴퓨터가 동시에 카드를 선택 → 상대 패턴 예측이 핵심
히스토리	이전 세트의 라운드별 선택 기록을 누적하여 Hard AI의 패턴 분석에 활용

▶ 기억력 게임 — 솔로 플레이

기억력 게임은 격자 보드에 뒤집힌 카드를 기억하여 같은 카드끼리 짝을 맞추는 1인 게임입니다. 난이도에 따라 보드 크기가 달라지며, Hard 모드에서는 짝을 맞출 때마다 새 카드가 추가되는 무한 진행 구조입니다.

난이도	보드 크기	특징
Easy	4 × 2 (8칸, 4쌍)	기본 구성, 모든 짝을 맞추면 클리어
Normal	5 × 2 (10칸, 5쌍)	기본 구성, 모든 짝을 맞추면 클리어
Hard	6 × 2 (초기)	짝 맞출 때마다 빈 슬롯에 덱에서 새 카드 리필 — 52장 소진까지 무한 진행

2.5 Hard 모드 알고리즘

▶ 블랙잭 Hard — 남은 카드 확률 기반 의사결정

블랙잭 Hard 모드의 딜러 AI는 단순한 점수 비교를 넘어, 현재 공개된 카드를 역산하여 텍에 남아있을 카드를 추정한 뒤 확률적으로 행동을 결정합니다.

[남은 카드 추정 — getPossibleRemainingCardsForDealer()]

전체 52장에서 플레이어 공개 카드, 딜러 공개 카드, 플레이어 히든카드를 제거하여 현재 텍에 남아있을 카드 목록을 계산합니다.

[3가지 확률 계산]

확률 지표	계산 방법	의미
safeProbability	버스트 없이 뽑을 수 있는 카드 수 / 전체 남은 카드 수	카드를 뽑아도 안 터질 확률
improveProbability	현재보다 21에 더 가까워지는 카드 수 / 전체	뽑았을 때 점수가 더 좋아질 확률
catchUpProbability	플레이어 점수 이상이 되는 카드 수 / 전체	뽑았을 때 플레이어를 추월할 확률

[딜러 점수 구간별 오픈 조건]

딜러 점수 구간	오픈 조건	설명
11 이하	safeProbability $\geq$ 0.9	점수가 낮으면 적극적으로 오픈
12 ~ 15	safeProbability $\geq$ 0.70 AND improveProbability $\geq$ 0.55	안전하고 개선 가능성 있을 때만
16 (playerScore $\geq$ 19)	safeProbability $\geq$ 0.45 AND improveProbability $\geq$ 0.35	상대가 높으면 위험을 감수
16 (playerScore $\geq$ 17)	safeProbability $\geq$ 0.55 AND improveProbability $\geq$ 0.45	중간 임계값
17 이상	safeProbability $\geq$ 0.55 AND catchUpProbability $\geq$ 0.30	추월 가능성 있을 때만 오픈

※ 딜러 점수가 플레이어 점수 이하인 경우 (이미 유리)에는 조건 없이 스톱

▶ 클라이밍 게임 Hard — 미니맥스 + 패턴 가중치

클라이밍 Hard AI는 두 가지 요소를 결합하여 카드를 선택합니다. 남은 라운드를 트리 탐색하는 미니맥스와, 이전 세트에서 누적한 플레이어 행동 패턴 기반 가중치입니다.

[미니맥스 탐색 구조]

함수	역할
getHardChoice()	모든 컴퓨터 카드 후보에 대해 evaluateComputerChoiceHard() 호출, 최선 선택
evaluateComputerChoiceHard()	컴퓨터 카드 하나를 고정 후 모든 플레이어 응수에 대해 solveWorstCaseHard() 호출
solveWorstCaseHard()	플레이어 관점에서 컴퓨터에게 가장 불리한 경우 선택 (최악의 경우 탐색)

※ 탐색 깊이(depth) = 현재 남은 라운드 수, 재귀적으로 마지막 라운드까지 탐색

[패턴 가중치 — getPatternWeight()]

세트가 끝날 때마다 saveCurrentSetHistory()를 통해 각 라운드(depth 0~3)에서 플레이어가 낸 카드를 playerHistoryByRound[4]에 누적합니다. Hard AI는 같은 라운드 위치에서 플레이어가 특정 카드를 낸 비율을 계산하여 가중치로 사용합니다. 비율이 높을수록 그 카드를 낼 가능성이 높다고 판단하여 미니맥스 점수에 가중치를 곱해 보정합니다.

요소	설명
playerHistoryByRound[4]	세트가 끝날 때마다 라운드별 플레이어 선택 카드를 누적 저장
패턴 비율 계산	history에서 해당 Role이 등장한 횟수 / 전체 횟수 → ratio
가중치 공식	baseWeight(1.0) + ratio × 0.6 (비율 < 0.5) 또는 + ratio × 1.0 (비율 ≥ 0.5)
weightedScore 계산	immediateScore × weight + future.weightedScore (재귀 누적)

▶ 기억력 게임 Hard — 무한 리필 보드

Hard 모드에서는 짝을 맞출 때마다 빈 슬롯 2개에 덱에서 새 카드를 가져와 리필합니다. 리필 전 remainingDeckCards를 체크하여 0장이 되면 리필을 중단하고 기존 짝들만으로 마무리합니다. 이를 통해 52장의 카드를 전부 사용할 때까지 게임이 이어지는 무한 진행 구조를 구현합니다.

3. 객체지향 요소 정리

OOP 요소	적용 위치	설명
추상 클래스	Game, Player, Dealer	순수 가상함수로 파생 클래스의 구현 강제
상속	Blackjack-MemoryGame-ClimbingGame : Game BlackjackDealer : Dealer : Player BlackjackPlayer : Player	단일 상속으로 코드 재사용 및 공통 인터페이스 공유
다형성	GameManager::games (vector<Game*>)	Game* 포인터로 세 게임 관리, game->play() 가상 함수 호출
오버라이딩	Blackjack-MemoryGame-ClimbingGame::play() BlackjackPlayer / BlackjackDealer::calculateScore() BlackjackDealer::playTurn()	부모의 가상 함수를 파생 클래스에서 재정의
오버로딩	Card::Card() / Card::Card(string, string)	생성자 오버로딩으로 기본 카드와 지정 카드 두 가지 생성 방식 지원
캡슐화	Card, Deck, Player, Game 등 전 클래스	private 멤버 + public 게터/세터로 내부 구현 은닉
네임스페이스	State, ConsoleHelper, InputHelper	관련 상수와 유틸 함수를 묶어 전역 이름 충돌 방지
열거형	State::Difficulty / PauseResult / GameResult ClimbingGame::Role / InputHelper::Key	의미 있는 상수 집합을 타입 안전하게 표현
내부 구조체	MemoryGame::Slot, MemoryGameView::SlotViewData ClimbingGame::MatchOutcome	게임 상태 표현용 경량 데이터 컨테이너
동적 할당	GameManager (생성자·소멸자)	Game* 벡터를 new로 생성, 소멸자에서 delete로 메모리 해제

4. 테스트 및 장애요인 해결 과정

문제 상황	원인	해결 방법
콘솔에서 한글·특수문자 깨짐	Windows 콘솔 기본 인코딩 문제	SetConsoleOutputCP(65001)으로 UTF-8 설정, ConsoleHelper에 공통 초기화 적용
화면 전체 갱신 시 깜박임 발생	cls로 전체를 지우고 다시 그리는 방식	SetConsoleCursorPosition으로 필요한 부분만 덮어써서 깜박임 최소화
블랙잭 Hard AI 과도한 강도	확률 threshold 초기값이 너무 낮게 설정됨	점수 구간별 safeProbability·improveProbability threshold 조정으로 밸런스 튜닝
MemoryGame Hard 모드 무한 루프	리필 시 남은 카드 0장 상황 미처리	리필 전 remainingDeckCards 체크 추가, 0장이면 리필 스킵
클라이밍 Hard AI 탐색 속도	미니맥스 깊이 제한 없이 전수 탐색	depth 제한으로 남은 라운드 수만큼만 탐색하도록 수정
게임 전환 시 난이도 초기화	각 게임 객체가 난이도를 독립 보관	GameManager에서 setDifficulty()를 모든 게임에 일괄 호출

5. 현재까지의 성과 및 결론

5.1 프로젝트 성과

항목	내용
구현 완성도	블랙잭·기억력 게임 완성, 클라이밍 게임 구조 설계 완료 (세부 구현 진행 중)
OOP 적용	추상 클래스 Game / Player / Dealer를 축으로 상속·다형성 구조 설계. 추상화·캡슐화·오버라이딩 등 수업 내용 직접 적용
AI 알고리즘	블랙잭: 남은 카드 확률 기반 3단계 의사결정 / 클라이밍: 미니맥스 + 세트 간 패턴 가중치 조합
멀티 요소	블랙잭 1대1 대전 (5라운드), 클라이밍 3세트 매치 구조로 대전 요소 구현
공통 구조	Card·Deck 클래스를 세 게임에서 공통 재사용, 게임 추가 시 확장이 용이한 구조

5.2 기대효과 및 활용방안

본 프로젝트는 트럼프 카드 한 덱을 기반으로 서로 다른 방식의 게임 3가지를 구현함으로써, 공통 객체 재사용과 게임별 로직 분리라는 객체지향 설계의 핵심 원칙을 직접 체득한 사례입니다. 특히 블랙잭의 확률 기반 AI와 클라이밍의 미니맥스+패턴 학습 AI는 단순한 게임 구현을 넘어 알고리즘 설계까지 경험하는 계기가 되었습니다. Game 추상 클래스를 상속하는 구조이므로 향후 새로운 카드 게임을 추가할 때 기존 코드를 수정하지 않고 확장이 가능하며, 멀티플레이 기능 확장이 가능하도록 구조를 설계하였으므로 네트워크 기능 추가나 GUI 환경으로 전환하는 후속 프로젝트로 발전시킬 수 있습니다.